

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 6
з дисципліни «Створення сервісу інвентаризації»

Виконав студент 2 курсу групи ФЕП-23

Накладюк Володимир Тарасович

Перевірив
асистент кафедри СП
Кулик Петро Русланович

Львів – 2026

Операційна система	Fedora 43
Репозиторій	https://github.com/Zefirok/backend-course-2025-6

Мета роботи

Метою лабораторної роботи є закріплення навичок роботи з HTTP-запитами, створення веб-сервісу на базі Express.js, реалізація операцій CRUD над колекцією об'єктів, тестування API через Postman та документування сервісу через Swagger. Додатково – освоєння обробки multipart/form-data за допомогою Multer і параметрів командного рядка через Commander.js.

Завдання роботи

У межах роботи потрібно було реалізувати сервіс інвентаризації, який:

- запускається з обов'язковими параметрами командного рядка --host, --port, --cache;
- реєструє нові інвентарні об'єкти з фото (POST /register);
- повертає список інвентаря та конкретний елемент (GET /inventory, GET /inventory/:id);
- оновлює назву, опис і фото (PUT /inventory/:id, PUT /inventory/:id/photo);
- видаляє об'єкт разом з його фото (DELETE /inventory/:id);
- містить HTML-форми RegisterForm.html та SearchForm.html;
- містить колекцію запитів Postman у файлі collection.json;
- має Swagger-документацію за маршрутом /api-docs;
- зафіксований у git мінімум 5 логічними комітами.

Хід виконання роботи

4.1. Створення проєкту та встановлення залежностей

Проєкт ініціалізовано в директорії lab6 за допомогою npm init. У package.json додано основні залежності та dev-залежності, а також npm-скрипти start і dev. Файл .gitignore містить виключення для node_modules, директорії cache та локальних файлів IDE.

Встановлені залежності:

- express – веб-фреймворк;

- commander – парсинг параметрів командного рядка;
- multer – обробка multipart/form-data (завантаження фото);
- swagger-ui-express – генерація Swagger UI;
- superagent – HTTP-клієнт (передбачений вимогами лабораторної);
- nodemon – автоматичний перезапуск при розробці (dev-залежність).

Команда встановлення:

```
npm install
```

[МІСЦЕ ДЛЯ СКРИНШОТА: структура проєкту у VS Code або терміналі з файлами проєкту]

Рисунок 1 – Структура створеного проєкту

4.2. Реалізація параметрів командного рядка

Для парсингу параметрів використано пакет Commander.js. Сервер приймає три обов'язкові параметри:

- -h, --host – адреса сервера;
- -p, --port – порт сервера;
- -c, --cache – шлях до директорії кешу фото.

Якщо хоча б один з параметрів не передано, програма виводить повідомлення про помилку і завершує роботу. Якщо директорії, переданої в --cache, не існує – вона створюється автоматично через fs.mkdirSync з опцією { recursive: true }.

Запуск сервера виконується командою:

```
node index.js --host 127.0.0.1 --port 3000 --cache ./cache
```

або через npm-скрипти:

```
npm start
npm run dev
```

[МІСЦЕ ДЛЯ СКРИНШОТА: запуск сервера з параметрами --host, --port і --cache у терміналі]

Рисунок 2 – Запуск сервера з параметрами командного рядка

[МІСЦЕ ДЛЯ СКРИНШОТА: приклад помилки при запуску без обов'язкового параметра]

Рисунок 3 – Перевірка обов'язкових параметрів

4.3. Реалізація WebAPI сервісу інвентаризації

У файлі index.js створено Express-додаток, який реєструє такі маршрути:

Метод	Маршрут	Призначення
POST	/register	реєстрація нового об'єкта (multipart/form-data)
GET	/inventory	список усіх інвентарних об'єктів
GET	/inventory/:id	отримання одного об'єкта за id
PUT	/inventory/:id	оновлення назви та/або опису
GET	/inventory/:id/photo	отримання фото об'єкта
PUT	/inventory/:id/photo	оновлення фото об'єкта
DELETE	/inventory/:id	видалення об'єкта і його фото
POST	/search	обробка HTML-форми пошуку

Інвентар зберігається у пам'яті сервера у вигляді JavaScript-масиву; id генерується автоінкрементом. Фото зберігаються у директорії, переданій через --cache, з унікальною назвою файлу (timestamp + випадкове число). Для будь-якого іншого HTTP-методу на відомих маршрутах, а також для невідомих маршрутів сервер повертає статус 405 Method Not Allowed.

[МІСЦЕ ДЛЯ СКРИНШОТА: фрагмент коду з основними маршрутами Express у редакторі]

Рисунок 4 – Реалізація маршрутів WebAPI

4.4. Реєстрація нового інвентаря

Маршрут POST /register приймає multipart/form-data з полями inventory_name, description та файлом photo. Завантажений файл обробляється посередником Multer, який зберігає його в кеш-директорії під унікальною

назвою. У разі успіху повертається статус 201 Created і JSON з даними створеного об'єкта (id, inventory_name, description, photo, photoUrl).

Якщо поле inventory_name відсутнє або порожнє, сервер повертає 400 Bad Request і видаляє завантажений файл, щоб не залишилось «осиротілого» фото у кеші.

[МІСЦЕ ДЛЯ СКРИНШОТА: успішний POST /register у Postman зі статусом 201]

Рисунок 5 – Реєстрація нового інвентаря через Postman

[МІСЦЕ ДЛЯ СКРИНШОТА: перевірка помилки 400 при відсутньому inventory_name]

Рисунок 6 – Обробка помилки при відсутній назві інвентаря

4.5. Отримання списку та конкретного елемента інвентаря

GET /inventory повертає JSON-масив усіх зареєстрованих об'єктів; для кожного – id, inventory_name, description і photoUrl (повне посилання на фото з урахуванням хоста і порту сервера).

GET /inventory/:id повертає JSON одного об'єкта за вказаним id. Якщо об'єкта з таким id не існує, повертається 404 Not Found.

[МІСЦЕ ДЛЯ СКРИНШОТА: GET /inventory у Postman зі списком інвентаря]

Рисунок 7 – Отримання списку інвентаря

[МІСЦЕ ДЛЯ СКРИНШОТА: GET /inventory/:id у Postman]

Рисунок 8 – Отримання інформації про конкретний елемент

4.6. Оновлення та видалення інвентаря

PUT /inventory/:id приймає JSON-тіло і дозволяє змінити поля inventory_name та/або description. Якщо id не знайдено – 404 Not Found, інакше – 200 OK з оновленим об'єктом.

DELETE /inventory/:id видаляє об'єкт зі списку і додатково видаляє відповідне фото з кеш-директорії через fs.unlink. Якщо id не знайдено – 404 Not Found.

[МІСЦЕ ДЛЯ СКРИНШОТА: PUT /inventory/:id у Postman]

Рисунок 9 – Оновлення інформації про інвентар

[МІСЦЕ ДЛЯ СКРИНШОТА: DELETE /inventory/:id у Postman]

Рисунок 10 – Видалення інвентаря

4.7. Робота з фото

Фото зберігаються у директорії, яка передається через параметр --cache. Для кожного завантаженого файлу формується унікальна назва вигляду <timestamp>-<rand>.jpg. Доступ до фото через HTTP здійснюється двома способами:

- за прямим URL /cache/<filename> (статична віддача);
- за маршрутом GET /inventory/:id/photo, який обов'язково встановлює заголовок Content-Type: image/jpeg і повертає файл як бінарну відповідь.

Якщо у запиті вказано неіснуючий id або у об'єкта немає фото, повертається 404 Not Found. Маршрут PUT /inventory/:id/photo приймає multipart/form-data з полем photo і замінює фото на нове, попередньо видаляючи старий файл.

[МІСЦЕ ДЛЯ СКРИНШОТА: GET /inventory/:id/photo у браузері або Postman]

Рисунок 11 – Отримання фото інвентаря

4.8. HTML-форми

У корені проекту створено два HTML-файли:

- RegisterForm.html – форма реєстрації нового інвентаря з полями inventory_name, description, photo, відправляє POST /register з enctype="multipart/form-data";
- SearchForm.html – форма пошуку інвентаря за id з полем id і чекбоксом has_photo, відправляє POST /search як application/x-www-form-urlencoded.

Маршрут POST /search приймає дані форми, шукає інвентар за id і повертає HTML-сторінку з інформацією про об'єкт. Якщо чекбокс has_photo увімкнено, у відповідь додається посилання на фото і вбудоване зображення. Якщо id не знайдено – 404 Not Found.

[МІСЦЕ ДЛЯ СКРИНШОТА: сторінка RegisterForm.html у браузері]

Рисунок 12 – HTML-форма реєстрації інвентаря

[МІСЦЕ ДЛЯ СКРИНШОТА: сторінка SearchForm.html у браузері]

Рисунок 13 – HTML-форма пошуку інвентаря

[МІСЦЕ ДЛЯ СКРИНШОТА: результат пошуку через SearchForm.html у браузері]

Рисунок 14 – Результат пошуку інвентаря через HTML-форму

4.9. Документування сервісу через Swagger

Документацію реалізовано через пакет swagger-ui-express і визначений у коді об'єкт swaggerDocument у форматі OpenAPI 3.0. У документації описано всі основні ендпоінти, формати тіл запитів (multipart/form-data, application/json, application/x-www-form-urlencoded), параметри шляху і можливі коди відповідей (200, 201, 400, 404). Swagger UI доступний за адресою:

<http://127.0.0.1:3000/api-docs>

[МІСЦЕ ДЛЯ СКРИНШОТА: Swagger UI зі списком ендпоінтів]

Рисунок 15 – Swagger-документація WebAPI

4.10. Тестування через Postman

Для тестування API створено колекцію у файлі collection.json формату Postman v2.1. До неї додано всі основні запити, а також два негативних кейси:

- Register item (POST /register);
- Bad request without inventory_name (POST /register без імені);
- Get all inventory (GET /inventory);
- Get inventory by ID (GET /inventory/:id);
- Not found by wrong ID (GET /inventory/9999);
- Update inventory (PUT /inventory/:id);
- Get photo (GET /inventory/:id/photo);
- Update photo (PUT /inventory/:id/photo);
- Search inventory (POST /search);
- Delete inventory (DELETE /inventory/:id);
- Register form / Search form (HTML).

Колекція використовує змінну baseUrl, за замовчуванням – <http://127.0.0.1:3000>. Файл collection.json доданий у репозиторій. Імпортується в Postman через Import → File → Upload Files.

[МІСЦЕ ДЛЯ СКРИНШОТА: Postman collection зі списком запитів]

Рисунок 16 – Колекція запитів у Postman

4.11. Git-коміти

Під час виконання роботи зроблено п'ять логічних комітів через командний рядок:

1. Initial project setup – створено package.json, .gitignore, package-lock.json;
2. Add command line arguments and server startup – реалізовано CLI-параметри та базовий запуск сервера;
3. Add inventory API endpoints – додано всі ендпоінти CRUD і завантаження фото;
4. Add HTML forms and search endpoint – додано RegisterForm.html, SearchForm.html і маршрут /search;
5. Add Swagger documentation and Postman collection – додано Swagger UI, файл collection.json і README.md.

[МІСЦЕ ДЛЯ СКРИНШОТА: результат команди git log --oneline з мінімум 5 комітами]

Рисунок 17 – Історія комітів проекту

Перевірка працездатності

Для перевірки роботи сервера використано браузер, утиліту curl і Postman. Результати перевірки наведено у таблиці:

Запит	Очікуваний результат
POST /register (валідний)	201 Created
POST /register без inventory_name	400 Bad Request
GET /inventory	200 OK
GET /inventory/:id	200 OK
GET /inventory/9999	404 Not Found
PUT /inventory/:id	200 OK
GET /inventory/:id/photo	200 OK, Content-Type: image/jpeg
PUT /inventory/:id/photo	200 OK
POST /search	200 OK
DELETE /inventory/:id	200 OK
DELETE /inventory (некоректний метод)	405 Method Not Allowed
GET /unknown (невідомий маршрут)	405 Method Not Allowed

Усі наведені сценарії перевірено локально – статус-коди відповідають очікуваням.

Висновок

Під час виконання лабораторної роботи було створено повноцінний веб-сервіс інвентаризації на базі Express.js. Реалізовано всі необхідні CRUD-операції над колекцією інвентаря, обробку завантаження і повернення фото через Multer, а також маршрут пошуку, який працює з HTML-формою. Додатково підготовлено дві HTML-форми (RegisterForm.html і SearchForm.html), документацію API через Swagger UI за маршрутом /api-docs і Postman-колекцію collection.json з усіма основними та негативними сценаріями. Тестування підтвердило коректність відповідей сервера: успішні запити повертають 200/201, невалідні – 400, ненайдені – 404, заборонені методи – 405. У результаті закріплено навички роботи з HTTP-запитами, маршрутизацією в Express, обробкою файлів через multipart/form-data, документуванням REST API і використанням Git для логічного поділу історії змін.